

Cognitive Robotics

Studienarbeit T3200

Studiengang Elektrotechnik
An der Dualen Hochschule Stuttgart
Campus Horb

Eingereicht von:

Name:	Luis Magnus Thiel	Adrian Sommer
Matrikelnr.:	8616207	9519924
Straße, Nr.:	Hugo-Bertsch-Straße 15	Hechinger Straße 40
PLZ, Ort:	72459 Albstadt	72336 Balingen

Partner:	ASSA ABLOY Sicherheitstechnik GmbH	Groz-Beckert KG
Adresse:	Bildstockstraße 20	Parkweg 2
Ort:	72458 Albstadt	72458 Albstadt

Studienjahrgang: TET2023

Bearbeitungszeitraum: 02. März 2026 bis 22. Juni 2026

Betreuer: Norman Bläse (Mercedes-Benz)

Kurzzusammenfassung

Diese Studienarbeit befasst sich mit der Optimierung und Erweiterung eines autonomen Robotersystems, bestehend aus einem Mitsubishi Movemaster EX RV-M1, einer 3D-ToF-Kamera sowie einer 2D-Industriekamera. Dabei ist das übergeordnete Ziel die Transformation des vorliegenden Aufbaus hin zu einem intelligenten System, welches durch die Implementierung von bildbasierten Sicherheitsfunktionen Potenziale für einen kollaborativen Betrieb aufzeigt.

Zunächst wird der Einfluss verschiedener Beleuchtungsbedingungen auf das Bildgewinnungssystem in einer Messreihe erforscht und mögliche Optimierungen erarbeitet. Nachfolgend gibt die Analyse der C++/C#-Softwarearchitektur Aufschluss über vorhandene Schwachstellen, wie auftretende Probleme bei der Thread-Synchronisierung und der Implementierung einer statischen Robotersteuerung. Für letzteres wird durch die Konzeptionierung einer dynamischen und geschlossenen Ansteuerung eine Alternative geschaffen.

Aufbauend auf den Erkenntnissen der Analyse und Konzeptionierung der dynamischen Ansteuerung wird ein Gesamtkonzept für die Entwicklung einer *Geschwindigkeits- und Abstandsüberwachung (SSM)* erstellt. Dieses Konzept nutzt die Fusion der 2D- und 3D-Bilddaten, um dynamische Warn- und Stoppbereiche softwareseitig zu realisieren. Die Arbeit bildet somit das Fundament für die sicherheitstechnische und steuerungstechnische Implementierung in der nachfolgenden Studienarbeit.

Abstract

This student research project deals with the optimisation and expansion of an autonomous robot system consisting of a Mitsubishi Movemaster EX RV-M1, a 3D ToF camera and a 2D industrial camera. The overarching goal is to transform the existing setup into an intelligent system that reveals the potential for collaborative operation through the implementation of image-based safety functions.

First, the influence of different lighting conditions on the image acquisition system is investigated in a series of measurements and possible optimisations are developed. Subsequently, the analysis of the C++/C# software architecture highlights existing weaknesses, such as problems occurring in thread synchronisation and the implementation of a static robot control system. For the latter, an alternative is created by designing a dynamic and closed-loop control system.

Based on the findings of the analysis and design of the dynamic control system, an overall concept for the development of a *Speed and Separation Monitoring (SSM)* system is created. This concept uses the fusion of 2D and 3D image data to implement dynamic warning and stop areas software-side. The work thus forms the foundation for the safety-related and control-related implementation in the subsequent student research project.

Selbständigkeitserklärung

Hiermit erklären wir, Luis Magnus Thiel und Adrian Sommer, dass wir diese schriftliche Studienarbeit selbständig verfasst haben. Es wurden keine anderen als die angegebenen Hilfsmittel und Quellen benutzt und alle wörtlich oder sinngemäß aus anderen Werken übernommenen Aussagen sind als solche gekennzeichnet.

Horb, den 19. Juni 2026

Ort, Datum



Unterschrift (Luis Magnus Thiel)

Horb, den 19. Juni 2026

Ort, Datum



Unterschrift (Adrian Sommer)

Inhaltsverzeichnis

1	Einleitung	1
2	Grundlagen	2
2.1	Stand der Technik: Safety-Implementation & Retrofitting .	2
2.1.1	kollaborative Robotik	2
2.1.2	Visuelle Handerkennung	4
2.1.2.1	Infrarot- und Wärmebildsensorik	4
2.1.2.2	2D- und 3D-Time-of-Flight-Scanner	5
2.1.2.3	Hautfarbenerkennung	6
2.1.3	KI-Handerkennungsmodell	7
2.1.4	Normen und Richtlinien	7
2.1.4.1	Rechtliche Grundlagen und funktionale Sicherheit	7
2.1.4.2	Sicherheitsanforderungen an kollaborative Robotersysteme	7
2.1.4.3	Sicherheitsgerichtete Anforderungen an optische Schutzsysteme	8
2.2	Bildverarbeitung	9
2.3	Grundlagen der Informatik	9
2.3.1	Producer-Consumer-Pattern	9
2.3.2	Task Parallel Library (TPL)	9
2.3.2.1	Task-based Asynchronous Pattern (TAP) .	10
2.3.2.2	Task Cancellation/kooperativer Abbruch .	10
3	Ausgangssituation/IST-Analyse	11
3.1	Architekturlimitationen des bisherigen Systems	11
3.1.1	Sequenzielle Befehlsstruktur	11
3.1.1.1	Theoretische Limitation	12
3.1.1.2	Messexperiment	13
3.1.1.3	Praktische Limitation - Laufzeitmessung mit Simulationsexperiment	15
4	Architekturoptimierung und -erweiterung	18
4.1	Konzeptionierung der dynamischen Robotersteuerung . .	18
4.1.1	Optionen zur Positionserkennung/Befehlsbewegungssynchronisation	18
4.2	Befehlswarteschlange für dynamischere Robotersteuerung	20

5	Implementation der Erweiterungen	21
5.1	Safetyimplementation	21
5.1.1	Kriterienbasierte Evaluierung der Sicherheitsüberwachungssysteme	21
5.1.2	Gegenüberstellung spezialisiertes KI-Modell und einfacher Objekterkennungsalgorithmus	26
5.2	Befehlspipeline	26
5.2.1	Vergleich von theoretischer und realer Performance	26
6	Fazit & Ausblick	27
	Literaturverzeichnis	vii
	Abkürzungsverzeichnis	ix
	Abbildungsverzeichnis	xii
	Erklärungen zur Arbeit und Hilfsmitteln	xiii

1 Einleitung

Die vorliegende Studienarbeit befasst sich im ersten Schritt mit der fundierten Erarbeitung der verwendeten methodischen und technologischen Grundlagen. Darauf aufbauend erfolgt eine detaillierte Analyse der Ausgangslage, welche explizit an die Ergebnisse und den Stand der vorherigen Studienarbeit anknüpft. Bestandteil dieser Evaluation sind eine umfassende Ist- und Laufzeitanalyse sowie die Ausarbeitung daraus abgeleiteter Optimierungsvorschläge.

Auf Basis dieser Erkenntnisse widmet sich die Arbeit der Konzeptionierung neuer Lösungsansätze. Ein zentraler Fokus liegt hierbei auf der Optimierung und Erweiterung der bestehenden Systemarchitektur. Dies beinhaltet insbesondere die effiziente Speicherung und Instanziierung von Punktwolken. Zur erweiterten visuellen Auswertung wird eine Funktion zur wahlweisen einfachen oder doppelten Anzeige der Punktwolken integriert, wobei der Wechsel zwischen diesen beiden Darstellungsmodi nutzerfreundlich über eine neu implementierte Auswahl Taste erfolgt.

Ein weiterer inhaltlicher Schwerpunkt ist die informationstechnische Umsetzung einer Befehlspipeline sowie relevanter Safety-Features. In diesem Zuge werden die zugrunde liegende Softwarestruktur und die angewandten Design-Patterns detailliert beleuchtet. Nach einer kritischen Abwägung und Einordnung verschiedener Ansätze zur Safety-Implementierung wird die Wahl der finalen Methode sachlich begründet und deren praktische Umsetzung dargelegt. Den Abschluss der Arbeit bildet ein Ausblick, der potenzielle Erweiterungen, gezielte Verbesserungsvorschläge und konkrete Anknüpfungspunkte für nachfolgende Studienarbeiten aufzeigt.

2 Grundlagen

2.1 Stand der Technik: Safety-Implementation & Retrofitting

2.1.1 kollaborative Robotik

Für die Modernisierung älterer Industrie- und Robotersysteme (Retrofitting) im Hinblick auf aktuelle Sicherheitsstandards und moderne Sensortechnik existieren vielfältige technologische Ansätze. Diese lassen sich anhand verschiedener Systemparameter kategorisieren. Ein grundlegendes Unterscheidungsmerkmal ist die Dimensionierung der Bilderfassung, bei der entweder kostengünstige 2D-Kameras oder tiefenauflösende 3D-Kamerasysteme eingesetzt werden. Zudem variiert die Systemarchitektur hinsichtlich der Kameraanzahl: Während Monokamerasysteme eine einfache Raumüberwachung ermöglichen, bieten Multikamerasysteme durch hardwareseitige Redundanz eine deutlich verlässlichere Objekterkennung sowie Schutz vor Verdeckungen im sensornahen Bereich.

Die algorithmische Auswertung lässt sich wiederum in zwei Philosophieansätze unterteilen: die gezielte Klassifizierung spezifischer Objekte wie menschlicher Gliedmaßen oder die universelle Erfassung jeglicher Fremdoobjekte innerhalb eines definierten Raumbereichs. Ergänzend oder alternativ zu visuellen Kamerasystemen kommen in der Praxis berührungslose Sensoren auf kapazitiver, induktiver oder laserbasierter Basis (z.B. Laserscanner) zum Einsatz.

Die informationstechnische Absicherung des Arbeitsraumes erfolgt je nach Systemkomplexität über die Festlegung einer einzigen, starren Verbotszone oder über die Aufteilung in mehrere, flexible Zonen. Letzteres ermöglicht die Implementierung einer dynamischen Geschwindigkeitsregelung (Dynamic Speed Control), bei der die Systemgeschwindigkeit adaptiv an die Position des Menschen angepasst wird. Diese Verbots- und Schutzräume können entweder statisch im Raum verankert oder dynamisch an die aktuelle Kinematik sowie den Bremsweg der Anlage gekoppelt werden, wie es unter anderem in Konzepten des Fraunhofer-Instituts [1] realisiert wird.

Ein zentrales wissenschaftliches Fundament für solche flexiblen Sicherheitsarchitekturen beschreiben Bdiwi, M., Pfeifer, M. und Sterzing, A. [2], die sich intensiv mit unterschiedlichen Aufgabenszenarien der Mensch-Roboter-Kollaboration (MRK) befassen. Die Autoren schlagen eine systematische Taxonomie vor, welche die Interaktion zwischen Mensch und Maschine in vier aufeinander aufbauende Stufen gliedert:

**Level 1: Geteilter Raum
(Sicherheitsgerichteter Stopp)**

**Level 2: Gemeinsame Aufgabe
(Speed & Separation Monitoring)**

**Level 3: Handing-Over
(Hybride Steuerung / Dynamik)**

**Level 4: Physische Interaktion
(Kraftregelung / Guiding)**

- **1. Geteilter Arbeitsraum:** In diesem Szenario arbeiten Mensch und Roboter räumlich nah beieinander, führen jedoch strikt getrennte Aufgaben in exakt zugewiesenen Zonen aus. Sobald der Mensch die virtuelle Grenze zur Roboterzone überschreitet, detektiert die Sensorik eine Schutzraumverletzung und löst unverzüglich einen Sicherheitsstopp der Anlage aus.
- **2. Gemeinsame Aufgabe ohne physischen Kontakt:** Hier agieren beide Akteure innerhalb einer vordefinierten Kooperationszone an einem gemeinsamen Prozess. Anstelle eines abrupten Halts wird die Geschwindigkeit des Roboters in Abhängigkeit vom realen Abstand zum Menschen kontinuierlich gedrosselt.
- **3. Bauteilübergabe (*Handing-over*):** Dieses Level beschreibt den direkten Transfer von Werkzeugen oder Werkstücken zwischen Mensch und Roboter. Eine hybride Steuerung sorgt in dieser Phase dafür, dass der Manipulator dynamisch und feinfühlig auf die Handbewegungen des Menschen im freien Raum reagiert.
- **4. Gemeinsame Aufgabe mit physischer Interaktion:** Auf der höchsten Kooperationsstufe ist ein direkter Körperkontakt zwin-

gend erforderlich. Ein Praxisbeispiel hierfür ist die manuelle Führung eines Schwerlastroboters durch menschliche Handkräfte, um schwere Bauteile präzise zu positionieren.

Die technische Umsetzung der Überwachung innerhalb dieser Interaktionsstufen stützt sich im Wesentlichen auf drei Säulen: die permanente Überwachung der internen Roboterparameter (wie Position, Achsgeschwindigkeiten und Kräfte), eine automatisierte Ereigniserkennung zur Identifikation kritischer Zustände (z.B. die visuelle Verdeckung eines Menschen im Sensorschatten) sowie eine kamerabasierte Nahfeldderkennung. Letztere dient dazu, im direkten Umfeld des Endeffektors sowohl die sensorische Bereitschaft des Bedieners (z.B. via Gesichtserkennung) als auch die exakte Lage von Körperteilen zu erfassen, um Verletzungen oder Quetschungen effektiv zu verhindern.

2.1.2 Visuelle Handerkennung

In diesem Kapitel werden verschiedene informationstechnische Ansätze, Methoden und Praktiken zur visuellen beziehungsweise optischen Handerkennung detailliert beleuchtet. Die theoretische Auseinandersetzung mit diesen technologischen Konzepten ist im Rahmen dieser Studienarbeit von zentraler Bedeutung, um die dedizierte Auswahl des in dieser Arbeit implementierten Verfahrens wissenschaftlich und methodisch zu begründen. Zu diesem Zweck werden im Folgenden die Vor- und Nachteile der jeweiligen Herangehensweisen systematisch evaluiert.

2.1.2.1 Infrarot- und Wärmebildsensorik

Ein etablierter Ansatz zur Detektion des menschlichen Körpers basiert auf der Nutzung von Passiv-Infrarot-Sensoren (PIR), welche im infraroten Spektralbereich operieren. Diese Systeme messen die emittierte Wärmestrahlung von Objekten, die sich innerhalb des definierten Erfassungsbereichs befinden. Mittels spezieller Lernalgorithmen, die auf verschiedenen empirischen Fallstudien basieren, gelingt auf dieser Grundlage die selektive Erkennung menschlicher, warmer Haut [3].

Als wesentlicher Vorteil dieses sensorischen Prinzips erweist sich die robuste Einsetzbarkeit in rauen, industriell geprägten Produktionsumgebungen, da das Messprinzip weitgehend unabhängig von sichtbaren transienten Lichtverhältnissen arbeitet. Demgegenüber stehen jedoch signifikante Nachteile: Neben der Notwendigkeit kostenintensiver Infrarotsensoren neigt das System zu

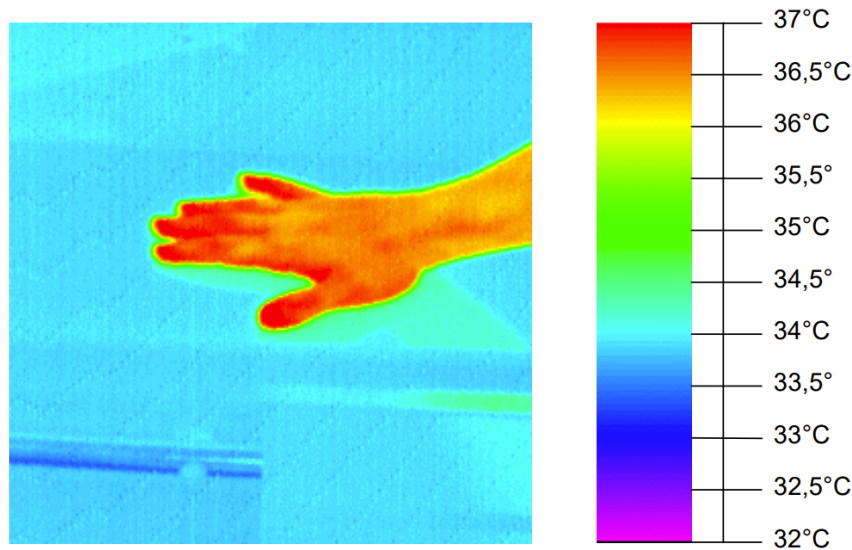


Abbildung 2.1: Testbild Passiv-Infrarot Schutzeinrichtung an einer Kreissäge aus [3]

Fehl- und Pseudoerkennungen, sobald die Temperatur umliegender Maschinenteile, Werkstücke oder Betriebsmedien der menschlichen Hautoberflächentemperatur ähnelt. Darüber hinaus erfordert die thermografische Signalverarbeitung und die nachgelagerte algorithmische Merkmalsberechnung eine vergleichsweise hohe Rechenleistung.

2.1.2.2 2D- und 3D-Time-of-Flight-Scanner

Andere wissenschaftliche Arbeiten, wie die Forschungen von Wolfgang Schulz [4], behandeln die Detektion von Handkonturen primär auf Basis von Abstandsaufnahmen. Technologisch lässt sich dieses Prinzip entweder als „virtueller Vorhang“ mittels einer planar messenden 2D-Time-of-Flight-Kamera (ToF) [4] oder als volumetrischer Scan des kompletten Arbeitsbereichs mithilfe einer 3D-ToF-Kamera realisieren, wie es unter anderem von Mandischer et al. [5] demonstriert wurde. Ein solches volumetrisches System ist in der Lage, einen großflächigen Arbeitsbereich von etwa 2×2 m lückenlos abzudecken.

Mithilfe der ToF-Struktur wird kontinuierlich die euklidische Distanz zwischen der Roboterkinematik und den im Raum befindlichen Objekten ermittelt. Unterschreitet der gemessene Abstand zwischen dem Manipulator und einem externen Objekt normative Schwellenwerte, greift ein algorithmisches Geschwindigkeits- und Abstandsmodell (*Speed and Separation Monitoring*, SSM), um den Roboter kontrolliert zu verlangsamen oder vollständig stillzusetzen.

Die Vorteile dieser Technologie liegen in der Option, eine mitbewegte Sensorik zu implementieren; das Kamerasystem kann hierbei direkt an der mechanischen Struktur des Roboters fixiert werden und wandert mit dem Gefahrenbereich mit. Zudem ermöglicht die inhärente Kombination aus Intensitätsbildern und dreidimensionalen Tiefendaten eine zuverlässige Positionsermittlung sowie die Differenzierung zwischen einem maschinellen Werkstück und einem menschlichen Körperteil.

Als kritischer Nachteil erweist sich jedoch die normative Zertifizierung im Kontext der funktionalen Sicherheit. Damit eine ToF-Kamera als berührungslos wirkende Schutzeinrichtung (ESPE) zugelassen werden kann, erfordert die Systemarchitektur eine hardwarenahe Zweikanaligkeit (Redundanz) sowie eine permanente, fehlersichere Selbstüberwachung. Zudem bedingt die hochfrequente Verarbeitung großer 3D-Punktwolken einen erheblichen Rechenaufwand, was zu systemischen Latenzzeiten in der Sicherheitskette führen kann.

2.1.2.3 Hautfarbenerkennung

Eine alternative Methode stellt die Extraktion und Erkennung menschlicher Körperteile auf Basis von rein farbmtrischen Informationen dar. Entsprechende Algorithmen nutzen Schwellwertverfahren, Histogrammanalysen oder vordefinierte Look-up-Tables (LUT), um bestimmte Farbwerte im Bild zu isolieren. Diese Analysen werden in unterschiedlichen Farbräumen wie RGB, HSV oder YCbCr durchgeführt. Ein kombinierter Ansatz, welcher die mathematischen Vorteile der Farbräume RGB, HSV und YCbCr kombiniert, wird in [6] beschrieben. Durch den sequentiellen Einsatz verschiedener Spektralfilter und komplexer Segmentierungsalgorithmen kann auf diese Weise eine robuste Extraktion der Handgeometrie oder anderer freiliegender Extremitäten erreicht werden.

Der fundamentale Vorteil dieser Methode liegt in der Hardware-Ökonomie, da für die Umsetzung eine klassische, kostengünstige 2D-Industriekamera ausreichend ist. Zudem arbeitet die farbbasierte Segmentierung weitgehend unabhängig von der konkreten Bewegung, Rotation oder temporären geometrischen Verformung der menschlichen Hand.

Dem stehen im industriellen Einsatz gravierende Nachteile gegenüber. Das System versagt inhärent, sobald der Werker vorgeschriebene Arbeitshandschuhe trägt, da die Farbinformation der Haut maskiert wird. Zudem verursachen die mathematischen Transformationen zwischen den Farbräumen und die nachgelagerte Segmentierung einen hohen theoretischen sowie rechentechnischen Aufwand. Das Verfahren setzt qualitativ hochwertige Kamerasysteme voraus und erfordert

eine präzise Kalibrierung auf das jeweilige optische System. Zuletzt begründet die strikte Abhängigkeit von den visuellen Hintergrundverhältnissen sowie wechselnden Beleuchtungsszenarien ein hohes Risiko für Falschdetektionen im praktischen Betrieb.

2.1.3 KI-Handerkennungsmodell

2.1.4 Normen und Richtlinien

Die Auslegung von Systemen der Mensch-Roboter-Kollaboration (MRK) sowie der Einsatz optischer Schutzsysteme unterliegen einem strikten regulatorischen Rahmen. Da bei kollaborativen Arbeitsplätzen die klassischen physisch-trennenden Schutzeinrichtungen (wie Schutzzäune) entfallen, muss die Sicherheit über sensorisch aufgenommene und durch spezielle Logik- verarbeitete Daten gewährleistet werden.

2.1.4.1 Rechtliche Grundlagen und funktionale Sicherheit

Die primäre rechtliche Basis für das Inverkehrbringen sowie wesentliche Veränderungen (Retrofitting) von Maschinenanlagen im europäischen Raum bildet die **Maschinenrichtlinie 2006/42/EG** [7] (bzw. die EU-Maschinenverordnung 2023/1230) [8]. Sie verpflichtet den Systemintegrator zu einer umfassenden Risikobeurteilung. Die steuerungstechnische Umsetzung des Sicherheitsnachweises wird durch die **DIN EN ISO 13849-1** [9] definiert. Diese Norm klassifiziert die Zuverlässigkeit sicherheits-bezogener Teile von Steuerungen anhand von fünf Performance Levels (PL_a bis PL_e). Für industrielle Roboterzellen im kooperativen Betrieb wird im Regelfall ein Mindest-Sicherheitsintegritätslevel von **PL d (Kategorie 3)** gefordert. Dies bedingt eine zweikanalige Struktur (Hardware-Redundanz) um, wie bereits in der vorherigen Studienarbeit erwähnt, die geforderte Ein-Fehler-Sicherheit zu gewährleisten.

2.1.4.2 Sicherheitsanforderungen an kollaborative Robotersysteme

Die allgemeinen Sicherheitsanforderungen für Industrieroboter sind in der Normenreihe **DIN EN ISO 10218-1/-2** [10] [11] verankert. Normativ werden vier Schutzprinzipien für die Mensch-Roboter-Interaktion definiert:

1. Sicherheitsgerichteter überwachter Halt (*Safety-rated monitored stop*)
2. Handführung (*Hand guiding*)

3. Geschwindigkeits- und Abstandsüberwachung (*Speed and Separation Monitoring*, SSM)

4. Kraft- und Leistungsbegrenzung (*Power and Force Limiting*, PFL)

Beim Einsatz berührungsloser Raumüberwachungssysteme ist das Prinzip des Speed and Separation Monitoring (SSM) von zentraler Bedeutung. Die **ISO/TS 15066** [12] liefert hierzu die mathematische Grundlage zur Berechnung des Mindestsicherheitsabstands S_p zwischen Mensch und Roboter:

$$S_p = v_H \cdot (T_R + T_B) + v_R \cdot T_R + s_R + C$$

In diese Berechnung fließen die menschliche Annäherungsgeschwindigkeit v_H , die Systemlatenzzeiten (Reaktionszeit der Sensorik T_R und Bremszeit des Roboters T_B) sowie gerätespezifische Nachlaufwege s_R und sensorische Zuschläge C ein.

2.1.4.3 Sicherheitsgerichtete Anforderungen an optische Schutzsysteme

Wird der Schutzraum volumetrisch überwacht, greift die Normenreihe **DIN EN IEC 61496-1** [13], welche die Anforderungen an berührungslos wirkende Schutzeinrichtungen (ESPE) regelt. Für den Einsatz in sicherheitsgerichteten Applikationen müssen optische Systeme als ESPE des Typs 3 oder Typs 4 klassifiziert sein. Dies setzt voraus, dass das Sensorsystem über eine permanente, hardwarenahe Selbstüberwachung (z.B. zur Erkennung von Linsenverschmutzungen, Signalverlusten oder internen Rechenfehlern) verfügt. Standardmäßige, einkanale Industriekameras ohne inhärente Redundanz sind für die Einleitung sicherheitsgerichteter Reaktionen normativ unzulässig.

Ergänzend definiert die **DIN EN ISO 13855** [14] die Anordnung von Schutzeinrichtungen im Hinblick auf die Annäherungsgeschwindigkeit des menschlichen Körpers. Sobald ein System auf die Detektion von Teilbereichen des Körpers, wie Hände oder Finger, ausgelegt wird, fordert die Norm den Ansatz standardisierter, maximaler Handgeschwindigkeiten (1600 mm/s bis 2000 mm/s). Für optische Systeme, die auf Mustererkennung basieren, ergibt sich daraus theoretisch die Anforderung, dass die Verarbeitungsfrequenz (*Frames per Second*) und die Pipeline-Latenz so dimensioniert sein müssen, dass eine Handbewegung innerhalb dieser Dynamikgrenzen deterministisch und ohne unzulässige Zeitverzögerung erfasst werden kann.

Zusammenfassend ergibt sich in der mathematisch-deterministischen Validierung nach ISO 13849-1 [9] und IEC 61496 [13] eine theoretische Diskrepanz beim Einsatz stochastischer

Auswertelgorithmen (wie maschinellen Lernverfahren). Da diese systembedingt keine klassischen deterministischen Zustandstabellen aufweisen, fordert die normative Praxis bei sicherheitskritischen Anwendungen stets eine Absicherung der mathematischen Ergebnisse durch diversitäre Sensorik oder fest programmierte Hardware-Gateways.

2.2 Bildverarbeitung

Filter, Algorithmen, Linien-/Kantendetektion mit OpenCV

2.3 Grundlagen der Informatik

2.3.1 Producer-Consumer-Pattern

2.3.2 Task Parallel Library (TPL)

Die *Task Parallel Library (TPL)* ist eine .NET-Bibliothek, die es dem Programmierer vereinfacht, Parallelität und Nebenläufigkeit in C#-Programmen umzusetzen [15]. Für diese Arbeit ist vor allem das Konzept der Nebenläufigkeit (*engl.: concurrency*) relevant. Darunter versteht man allgemein, dass "mehr als nur eine Sache zur gleichen Zeit gemacht wird" [16, Kapitel 1]. In der praktischen Anwendung ermöglicht das eine Entkopplung von Rechenoperationen oder Ein-/Ausgabeoperationen vom primären Arbeitsstrang einer Anwendung. In Abbildung 2.2 werden synchrone und asynchrone

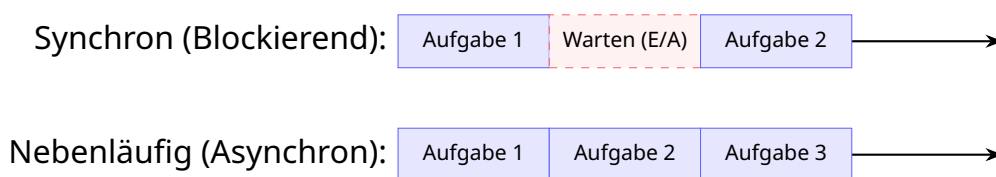


Abbildung 2.2: Problematik von blockierenden Abläufen

Abläufe gegenübergestellt. Es lässt sich erkennen, dass bei einem synchronen Ablauf eine Wartezeit den aufrufenden Thread blockiert - TPL löst diese Problematik durch die Einführung von asynchronen *Tasks*.

Nebenläufigkeit vs. Parallelität

Um die Rolle der TPL im Kontext dieser Arbeit zu präzisieren, muss zwischen Nebenläufigkeit und Parallelität unterschieden

werden. Während Parallelität die gleichzeitige Ausführung mehrerer Aufgaben auf verschiedenen Prozessorkernen beschreibt, definiert Nebenläufigkeit die organisatorische Strukturierung eines Programms. Ein nebenläufiges System ist darauf ausgelegt, Aufgaben durch verschachtelten Fortschritt (*engl.: Interleaving*) so zu bearbeiten, dass eine Anwendung reaktionsfähig bleibt, während sie im Hintergrund andere Prozesse abarbeitet.

Dabei ist Multithreading – also die Verwendung mehrerer Ausführungsstränge – lediglich eine Form der nebenläufigen Programmierung. Moderne Frameworks wie die TPL bieten hierfür übergeordnete Abstraktionen, die den direkten, fehleranfälligen Umgang mit Threads durch effizientere Konzepte ersetzen.

Task-Modell

Die TPL löst die starre Bindung an Betriebssystem-Threads durch das Konzept des Tasks (.NET-Bibliothek `System.Threading.Tasks`) auf. Ein Task repräsentiert eine asynchrone Operation und verkörpert das Entwurfsmuster eines Futures oder Promises – ein logisches Versprechen auf ein in der Zukunft liegendes Ergebnis. Anstatt Aufgaben fest an schwerfällige Betriebssystem-Threads zu binden, entkoppelt die TPL die logische Aufgabe von der physikalischen Ausführung. Ein interner Thread-Pool verwaltet eine minimale Anzahl von Arbeiter-Threads und verteilt die anfallenden Tasks dynamisch. Wartet ein Task beispielsweise auf eine Netzwerkanfrage, gibt er den physischen Thread sofort für andere nebenläufige Aufgaben frei.

2.3.2.1 Task-based Asynchronous Pattern (TAP)

Das Task-based Asynchronous Pattern (TAP) setzt das zuvor beschriebene Task-Modell ab .NET-Framework 4 als Standard für die asynchrone Programmierung in C# durch [17]. Hierzu kommen die Schlüsselwörter **async** und **await** zum Einsatz.

2.3.2.2 Task Cancellation/kooperativer Abbruch

3 Ausgangssituation/IST-Analyse

3.1 Architekturlimitationen des bisherigen Systems

3.1.1 Sequenzielle Befehlsstruktur

Die vorliegende Befehlsübermittlung an den Movemaster EX RV-M1 wird in der Software *Objekterkennung_RoboETH* sequenziell realisiert. ?? zeigt qualitativ auf, wie eine solche Abfolge aussieht. Es ist

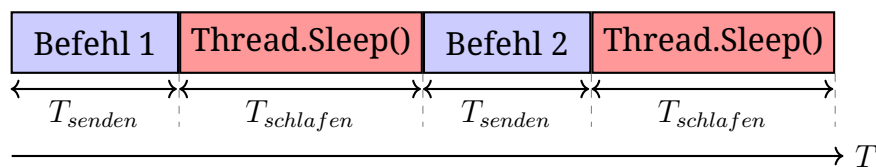


Abbildung 3.1: Sequenzielle Abfolge an Befehlen (Eigene Darstellung)

erkennbar, dass nach jedem gesendeten Befehl ein *Thread.Sleep()-Befehl* den gesamten Hauptthread für eine nicht vernachlässigbare Zeitspanne T_{schlafen} blockiert. Die theoretische Zeit T die benötigt wird, bis ein Befehl vollständig an den Roboter gesendet, von ihm verarbeitet und ausgeführt wird, lässt sich vereinfacht mit $T_{\text{befehlsausfuehrung}} = T_{\text{senden}} + T_{\text{schlafen}}$ beschreiben.

Während dieser Zeitspanne friert die gesamte Benutzeroberfläche sowie die Handlungsfähigkeit des Hauptthreads ein. Um die Auswirkungen der sequenziellen Befehlsbereitstellung auf die Gesamtlaufzeit und die Systemstabilität zu quantifizieren, ist eine detaillierte Analyse der Ausführungszeiten erforderlich. Hierfür wird in 3.1.1.2 ein reproduzierbares Messexperiment aufgesetzt.

Das Ziel dieser Analyse ist die Identifikation der kritischen Pfade. Durch eine explizite Messung der Zeitstempel für den Eintritt in die Befehlsverarbeitung, die Dauer der Hardware-Kommunikation und die Verweildauer in den Wartezuständen wird der „Flaschenhals“ der synchronen Architektur isoliert. Erst durch diese messtechnische

Herleitung wird transparent, an welcher Stelle die CPU-Last ungenutzt bleibt und wie sich die Thread-Auslastung sowie die Laufzeit bei einer Umstellung auf eine asynchrone Pipeline-Architektur verbessern lässt.

3.1.1.1 Theoretische Limitation

Wird erneut Abbildung 3.1 betrachtet, so fällt auf, dass die Zeit, welche benötigt wird, um die Koordinaten der einzelnen Objekte im Arbeitsbereich des Roboters zu berechnen, nicht eingerechnet wird. Grund hierfür ist die Annahme, dass die Berechnungszeit in beiden Fällen (sequenzielle Befehlsstruktur oder asynchrone Befehlspipeline) voraussichtlich gleich ist und im Vergleich mit den Ausführungszeiten der physischen Roboterbewegung vernachlässigbar ist.

Die theoretische Gesamtlaufzeit T_s einer Befehlssequenz lässt sich somit durch Summation der Befehlsausführungszeiten $T_{befehlsausfuehrung}$ bestimmen:

$$T_s = \sum_{i=1}^n \sum_{j=1}^k T_{befehlsausfuehrung,i,j} \quad (3.1)$$

Einsetzen der zuvor beschriebenen Zeiten ergibt:

$$T_s = \sum_{i=1}^n \sum_{j=1}^k (T_{senden,i,j} + T_{schlafen,i,j}) \quad (3.2)$$

Hierbei repräsentiert der Index i das jeweilige Objekt innerhalb der Bearbeitungsreihenfolge (mit n als Gesamtzahl der Objekte), während der Index j die spezifische Befehlsposition innerhalb eines Manipulationszyklus angibt (mit k als Anzahl der Befehle pro Objekt).

In den Gleichungen 3.1 und 3.2 lässt sich die direkte Korrelation zwischen dem Parameter n und der durch die Bildverarbeitungsklasse erfassten Objekte erkennen. Des Weiteren wird verdeutlicht, dass die Gesamtausführungszeit einer Befehlssequenz (bestehend aus $1, \dots, k$ Befehlen pro Objekt) linear von der Objektanzahl n und der aufsummierten Wartezeiten durch *Thread.Sleep()*-Befehlen abhängt.

Bei dieser theoretischen Betrachtung wird bewusst eine Worst-Case-Abschätzung mittels der fest definierten Wartezeiten getroffen, um sicherzustellen, dass der Roboter die Bewegung garantiert abgeschlossen hat, bevor der nächste Befehl gesendet wird. Dabei werden die Wartezeiten entweder auf Basis von experimenteller

Erfahrung oder durch grobe Berechnung der benötigten Zeit gesetzt. Praktisch funktioniert diese Vorgehensweise, allerdings können hieraus Einbußen in der Systemeffizienz und -dynamik resultieren.

3.1.1.2 Messexperiment

Um die theoretisch hergeleiteten Ineffizienzen und die zeitlichen Auswirkungen der sequenziellen Befehlerteilung quantitativ nachzuweisen, wurde ein deterministisches Messexperiment aufgesetzt. Selbiges muss für Vergleichbarkeit und Bewertbarkeit auch bei nachfolgenden Erweiterungen des Systems (siehe 4.2) zu Grunde gelegt werden.

Das Messexperiment wird mit vier Zylinderobjekten durchgeführt - die Positionen sind jeweils auf der Arbeitsunterlage markiert. Abbildung 3.2 zeigt die Anordnung der Objekte sowie die zugehörigen Objektkoordinaten. Die Geschwindigkeit des Roboters wird per Befehl



(a) Objektanordnung

Objekt-ID: X-, Y-, Z-, Roll, Pitch

Objekt 1: -42, 291, 6, -90, -8

Objekt 2: -37, 374, 6, -90, -5

Objekt 3: 36, 295, 6, -90, 6

Objekt 4: 55, 346, 6, -90, 9

(b) Koordinaten im Messexperiment

Abbildung 3.2: Messexperiment zur Evaluation der Sortierperformance

SP fest auf Stufe 5 H (0-9 Stufen, H = Hohe Beschleunigung) gesetzt. Sowohl die maximalen als auch die eingestellten Geschwindigkeiten der einzelnen Achsen lassen sich aus Tabelle 3.1 entnehmen und sind für die spätere Betrachtung der inversen Kinematik des Movemaster EX RV-M1 relevant.

Achse	maximale Geschwindigkeit	SP5-Geschwindigkeit
Joint 1	120 °/sec	29,64 °/sec
Joint 2	72 °/sec	17,78 °/sec
Joint 3	109 °/sec	26,92 °/sec
Joint 4	100 °/sec	24,7 °/sec
Joint 5	163 °/sec	40,26 °/sec

Tabelle 3.1: maximale bzw. SF5-Gelenkgeschwindigkeiten nach [18, APP-52, 1-7]

Das Ziel, alle Objekte jeweils in einen leeren Magazinplatz zu sortieren, kann in vier Objektsortierungssequenzen und zwei Sequenzen für die Start-/Endposition eingeteilt werden. Dabei enthält jede Objektsortierungssequenz Fahr- und Greifbefehle - ersichtlich in Abbildung 3.3.

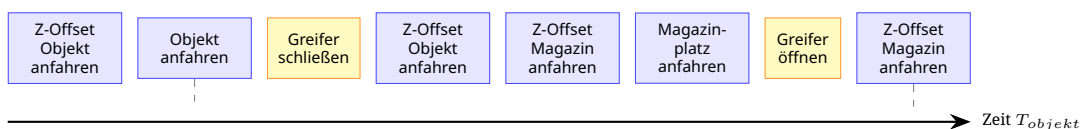


Abbildung 3.3: Befehle innerhalb einer Objektsortierungssequenz

3.1.1.3 Praktische Limitation - Laufzeitmessung mit Simulations-experiment

Um die praktische Limitation der bisherigen Architektur sinnvoll beurteilen zu können, wird zunächst die theoretisch benötigte Laufzeit mit Gleichung 3.2 ermittelt.

$$T_s = \sum_{i=1}^n \sum_{j=1}^k (T_{senden,i,j} + T_{schlafen,i,j})$$

Betrachtet man den Programmcode der Klasse *Roboter.cs*, Methode *SendData*, so fällt auf, dass jeder Sendevorgang ($T_{senden,i,j}$) mit einem `Thread.Sleep()` von 800ms beaufschlagt wird. Dies geschieht, um sicherzugehen, dass innerhalb der Befehlsweiterleitung von C#-Programm zu RoboETH und letztlich über RS232C an den Movemaster EX RV-M1 genügend Zeit für die RoboETH-Bridge vorhanden ist, um die Befehle zu verarbeiten und an den Roboter weiterzuleiten. Das verhindert Kommandoüberläufe sowie einen Absturz der Bridge und ermöglicht ein sicheres Empfangen über RS232C.

Um die festgelegte Zeit $T_{schlafen,i,j}$ für ein Objekt zu ermitteln, werden die Methoden *SendKoordinations*, *GoToMagazin* sowie *GoToStartPos* betrachtet und in der nachfolgenden Tabelle dargestellt.

Beschreibung	Thread.Sleep() [ms]	$T_{schlafen,i,j}$
Fahrt auf Objekt Z-Offset	3000	$T_{schlafen,i,1}$
Absenken auf Objektposition	3000	$T_{schlafen,i,2}$
Objekt greifen	500	$T_{schlafen,i,3}$
Anheben auf Objekt Z-Offset	2000	$T_{schlafen,i,4}$
Fahren auf Magazin Z-Offset	5000	$T_{schlafen,i,5}$
Absenken auf Magazinposition	1000	$T_{schlafen,i,6}$
Loslassen des Objekts	1000	$T_{schlafen,i,7}$
Anheben auf Magazin Z-Offset	1000	$T_{schlafen,i,8}$
<i>Anheben auf Magazin Z-Offset</i>	1000	$T_{schlafen,i,9}$

Tabelle 3.2: Tabellarische Übersicht der Thread.Sleep()-Verzögerung pro Objektsequenz

Setzt man nun die statischen Werte aus Tabelle 3.2, die Wartezeiten bei jedem Senden sowie die benötigte Zeit für das Erreichen von Start- und Endposition in die Gleichung für die Gesamtdauer T_S ein

$$T_S = T_s = 2 * (1800ms) + \sum_{i=1}^4 \sum_{j=1}^9 (T_{senden,i,j} + T_{schlafen,i,j}) \quad (3.3)$$

so erhält man eine berechnete Gesamtdauer von 102400 ms. Im Rahmen der Ist-Analyse der vorliegenden Architektur wurde mit der *Stopwatch*-Klasse im C#-Namensraum *System.Diagnostics* präzise gemessen, wie lange die gesamte softwareseitige Ausführung der Sortieraufgabe benötigt. Dies ist nachfolgend in 3.3 dargestellt.

Objekt-ID	Theoretische Zeit T_S [s]	Gemessene Zeit T_{mess} [s]
Startposition	1,8	2,01
Objekt 1	24,7	24,97
Objekt 2	24,7	24,91
Objekt 3	24,7	24,95
Objekt 4	24,7	24,89
Endposition	1,8	2,00
T_S	102,40	103,73

Tabelle 3.3: Vergleich der theoretischen Laufzeit T_S mit den gemessenen Werten T_{mess}

Beim Vergleich der berechneten und gemessenen Laufzeiten lässt sich erkennen, dass die aktuelle Architektur hinsichtlich des *Overheads* bereits sehr performant ist (Abweichung von 1,3% zwischen Berechnung und Messung).

Beurteilung des IST-Standes

Während der Durchführung des Messexperiments am aktuellen Softwarestand ließ sich erkennen, dass die Hardware-Anforderungen des Movemaster EX RV-M1 souverän gemeistert werden und der Sortierablauf zuverlässig erfolgt. Allerdings konnten visuell Stillstandszeiten ausgemacht werden, welche die Dynamik des Roboters vor allem bei folgenden Bewegungen stark einschränken:

- Absenken und Anheben des Roboters auf Z-Offset (über Objekt oder Magazin)
- Objektfahrten

- Magazinfahrten

Die Abschätzung hinsichtlich der benötigten Zeiten für das Greifen/Loslassen eines Objekts sowie die Fahrt zu Start-/Endposition wurde hingegen gut getroffen. Auf Basis dieser Erkenntnisse wird in Kapitel 4 ein SOLL-Stand der Architektur erarbeitet und aufgezeigt, wie eine mögliche Implementierung aussehen kann.

4 Architekturoptimierung und -erweiterung

4.1 Konzeptionierung der dynamischen Robotersteuerung

In diesem Kapitel werden Möglichkeiten erläutert, wie die Objektsortierung durch den Movemaster EX RV-M1 beschleunigt, bzw. in erster Linie dynamisiert werden kann. Unter Dynamisierung wird nachfolgend das weitflächige Eliminieren von blockierenden Thread.Sleep()-Methoden eingeordnet.

4.1.1 Optionen zur Positionserkennung/Befehls-Bewegungssynchronisation

Um eine dynamische und nahezu unterbrechungsfreie Befehlserteilung zu realisieren, ist ein Mechanismus unerlässlich, der der Steuerungssoftware mitteilt, wenn die physische Bewegung des Roboters vollständig abgeschlossen ist. Nachfolgend werden unterschiedliche Möglichkeiten betrachtet, mit denen ein solches *Acknowledgement* erzielt werden kann.

Ereignisbasiertes Hardware-Feedback

Ein übliches Verfahren zur Synchronisation des Zustandes der Steuerungssoftware mit der aktuellen Position des Roboters stellen ereignisbasierte Architekturen (event-driven architectures) dar. Hierbei sendet der Roboter bei Erreichen der Zielposition und -orientierung ein Quittierungssignal über den Kommunikationskanal zurück an die Steuerung. Innerhalb der Steuerung wird ein *Event-Handler* aktiviert, der den weiteren Ablauf einleitet. Allgemeine Vorteile eines solchen Systems sind nahezu sofortige Reaktionszeiten der Steuerung sowie niedrige Datenmengen [19].

Der Mitsubishi Movemaster EX RV-M1 bietet allerdings keine hardwareseitige Implementation einer solchen Rückmeldung. Die Drive Unit des Roboters quittiert den fehlerfreien Erhalt eines per RS232C gesendeten Befehl, liefert allerdings keine Auskunft über

den Abschluss der Roboterbewegung [18, Vgl.]. Eine ereignisbasierte Architektur über den Standard-Befehlssatz und der Standard-Hardware des EX RV-M1 ist somit ausgeschlossen.

Zyklische Zustandsabfrage mit 'WH'-Befehl

Eine alternative Methode zur Zustandserkennung stellt das zyklische Abfragen der Gelenk- oder Raumkoordinaten im Hintergrund (engl. *Active Polling*) dar. Parallel zur Bewegungsausführung sendet die Software kontinuierlich den Befehl *WH* (Where) an den Controller. Durch den Abgleich der aktuellen Ist-Koordinaten mit den Ziel-Soll-Koordinaten innerhalb eines definierten Toleranzschlauchs ($\|X_{\text{Ist}} - X_{\text{Soll}}\| < \epsilon$) kann der physikalische Abschluss der Bewegung mathematisch approximiert und der nächste Befehl freigegeben werden. In experimentellen Voruntersuchungen (siehe Abschnitt 3) erwies sich dieser Ansatz für den RV-M1 jedoch als **instabil und unpraktikabel**. Die serielle Kommunikation (bzw. die bridge-basierte TCP-Schnittstelle) läuft bei der hochfrequenten Interaktion aus Schreibbefehlen (Bahndaten) und Lesebefehlen (Polling) in eine *Schnittstellen-Saturierung* (Buffer Overflow). Da der Prozessor des Roboters zudem Rechenkapazitäten für das Parsen und Senden der Positionsdaten aufwenden muss, bricht die interpolierte Trajektorie ein, was sich in ruckartigen Bewegungen oder vollständigen Kommunikationsabbrüchen äußert.

Prädiktive Trajektorieberechnung mit inverser Kinematik

Da eine direkte sensorische oder softwareseitige Rückmeldung der Hardware fehlschlägt, wechselt die vorliegende Konzeption von einem reaktiven zu einem **prädiktiven (vorhersagenden) Steuerungsmodell** (engl. *Predictive Time-Scaling*). Anstatt auf ein Feedback der Hardware zu warten, berechnet die Software die physikalisch notwendige Dauer jeder Bewegung im Voraus, bevor der Befehl an den Puffer übermittelt wird. Da der *MP-Befehl* eine strikt achssynchrone PTP-Interpolation im Gelenkraum (Joint Space) erzwingt, bewegen sich alle Achsen gleichzeitig los und erreichen simultan das Ziel. Die Gesamtdauer wird folglich deterministisch durch die langsamste Achse – die sogenannte *Führungssachse* (engl. *Leading Axis*) – diktiert [?]. Über die analytische inverse Kinematik werden für jeden Zielpunkt die exakten Gelenkwinkeldifferenzen $\Delta\theta_i$ (mit $i = 1, \dots, 5$) berechnet. Unter Einbeziehung der spezifischen maximalen Winkelgeschwindigkeiten

$\dot{\theta}_{\max,i}$ und des systemweiten Override-Faktors ($v_{\text{override}} = 0,247$ für die Geschwindigkeitsstufe SP5) ergibt sich die prädiktive Fahrzeit zu:

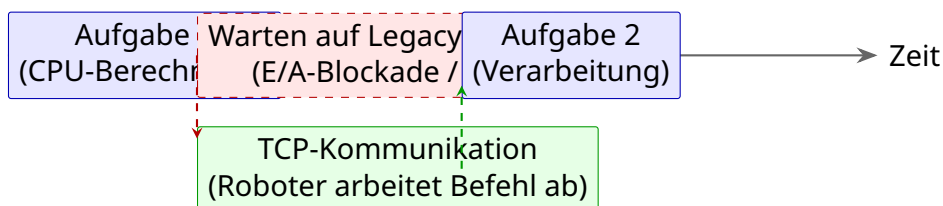
$$T_{\text{Fahrt}} = \max_{i=1}^5 \left(\frac{|\theta_{i,\text{Ziel}} - \theta_{i,\text{Start}}|}{\dot{\theta}_{\max,i} \cdot v_{\text{override}}} \right) \quad (4.1)$$

Diese berechnete Zeitspanne wird zuzüglich eines minimalen, empirisch ermittelten Rampenkoeffizienten als dynamische Blockadezeit (*TimeoutMs*) an die asynchrone Befehlspipeline übergeben. Da es sich um ein reines Softwaremodell handelt, wird die Hardwarechnittstelle des RV-M1 nicht belastet. Die statischen, konservativen *Worst-Case*-Wartezeiten des alten Systems schrumpfen somit dynamisch auf das mathematisch kleinstmögliche, pfadabhängige Minimum zusammen.

4.2 Befehlswarteschlange für dynamischere Robotersteuerung

Synchroner Ablauf (Blockierend):

Hauptthread ist während der E/A-Operation blockiert



Asynchroner Ablauf (Nebenläufig via TPL / TAP):

Thread-Freigabe bei E/A-Operationen dank async/await

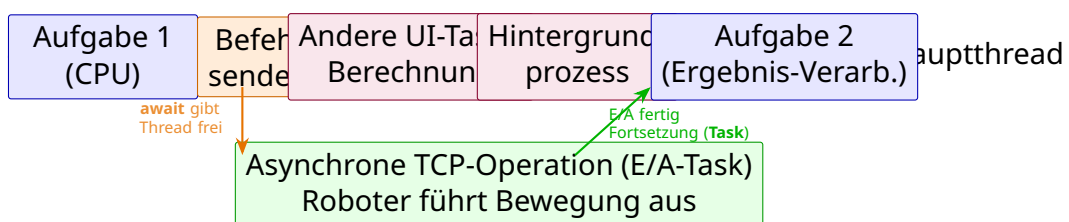


Abbildung 4.1: Gegenüberstellung von blockierenden synchronen Abläufen und dem ressourceneffizienten, asynchronen Task-basierten Muster (TAP) bei E/A-Operationen.

5 Implementation der Erweiterungen

Warteschlange, Safety, Punktwolkenspeicherung, ...

5.1 Safetyimplementation

5.1.1 Kriterienbasierte Evaluierung der Sicherheitsüberwachungssysteme

In der modernen industriellen Produktion hat sich eine Vorgehensweise etabliert, die auf einer systematischen Taxonomie der Aufgaben und Anwendungsfelder basiert. Dieser Ansatz ist maßgeblich, um funktionierende sowie finanziell und funktionell optimierte Absicherungssysteme zu entwickeln. Die Interaktion zwischen Mensch und Maschine wird in diesem Kontext standardmäßig als Human-Robot Interaction (HRI) beziehungsweise im industriellen Umfeld als Mensch-Roboter-Kollaboration (MRK) abgekürzt.

Die Auswahl einer geeigneten MRK-Sicherheitsüberwachung stützt sich auf diverse technologische und funktionale Entscheidungskriterien. Diese werden in der folgenden Evaluierungsmatrix anhand zentraler Kategorien – wie dem Aufbau des Arbeitsbereichs, dem den Notzustand einleitenden Ereignis, der eingesetzten Sensorik sowie der resultierenden Systemreaktion des Roboters – gegenübergestellt. Um den Rahmen dieser Arbeit zu wahren, konzentriert sich die Darstellung für jede Kategorie exemplarisch auf eine Gegenüberstellung zweier primärer Ausprägungen (Variante A und Variante B), wenngleich in der Praxis weitere technologische Mischformen existieren.

Für die informationstechnische Abgrenzung des Kollaborationsraumes stehen sich statische und dynamische Konzepte gegenüber. Ein statischer Sicherheitsbereich zeichnet sich primär durch die Einfachheit seiner Umsetzung aus. Die Begrenzung kann entweder über einen physischen, kontrastreichen Rahmen im realen Hintergrund des Arbeitsbereichs oder rein softwareseitig über die Definition eines virtuellen Rahmens (Region of Interest, ROI) innerhalb der Bildmatrix realisiert werden. Eine Kaskadierung des Raumes in mehrere Teilzonen ist zudem möglich und stellt das grundlegende Fundament für die

Kriterium	Variante A	Variante B
Sicherheitsbereich	statisch	dynamisch
Ereignis	Objekterkennung	Handerkennung
Sensorik	physisch	visuell
Aktion	Nothalt	Speed and Separation-Prinzip

Tabelle 5.1: Entscheidungsmatrix zur Auswahl der MRK-Sicherheitsüberwachung

Implementierung des Speed and Separation Monitoring (SSM) dar. Die für diese Studienarbeit relevanteste Methode nutzt das definierte Blickfeld (Field of View, FoV) einer 2D-Kamera als feste Begrenzungsebene. Erkennt der Algorithmus innerhalb dieses Sichtfeldes eine Verletzung der definierten Grenzen durch eine Bedrohung, wird die Sicherheitsreaktion des Roboters initiiert. Da der Sichtbereich der Kamera hierbei als absolute Grenze fungiert, spielt die exakte geometrische Ausrichtung auf den zu betrachtenden Arbeitsraum eine prozesskritische Rolle.

Demgegenüber passt sich ein dynamischer Sicherheitsbereich adaptiv in Echtzeit an die aktuelle Pose des Roboterarms an. Die Komplexität dieser Implementierung ist bedeutend höher als bei statischen Systemen, da für die Hüllkurvenberechnung permanent die Vorwärtskinematik des Roboters gelöst und die Transformation der Koordinatensysteme in den dreidimensionalen Raum berechnet werden müssen. Dies bedingt eine sehr geringe Datenbus-Latenz (Echtzeitfähigkeit), stellt hohe Anforderungen an die Rechenleistung des Gesamtsystems und erfordert für den Werker eine dynamische, visuelle Anzeige des aktiven Bereichs, beispielsweise über Lichtprojektionen oder innerhalb eines Digitalen Zwillings. Der technologische Mehraufwand resultiert jedoch in einer maximalen Interaktionsfreiheit zwischen Mensch und Roboter und bietet eine hervorragende Kombinierbarkeit mit dem SSM-Prinzip.

Das Fraunhofer-Institut für Fabrikbetrieb und -automatisierung IFF hat genau eine solche Technologie zur optischen Arbeitsraumüberwachung patentieren lassen. Die Entwickler setzen auf einen lichtprojizierten Sicherheitsrahmen welcher von der Bildverarbeitung als Kontrast erkannt und bei Unterbrechungen ein Nothalt des Roboters veranlasst wird. Der auf den Hintergrund projizierte Rahmen bietet den Vorteil, dass sich seine Position parallel zum Roboter anpassen lässt. In Abbildung 5.1 ist eine visuelle Veranschaulichung des Prinzips

geboten.

Bei der algorithmischen Ereignissteuerung wird zwischen universeller Objekterkennung und spezifischer Handerkennung unterschieden. Dient eine generelle Objekterkennung als auslösendes Ereignis für einen Sicherheitsstopp oder eine Geschwindigkeitsreduzierung, basiert dies meist auf klassischer Kantenextraktion oder Hintergrundsubtraktion.



Abbildung 5.1: Projektions- und kamerabasierte Technologie zur Arbeitsraumüberwachung

Sobald ein Fremdobjekt im Arbeitsbereich detektiert wird, hält die Anlage an. Ein inhärentes Problem dieses Ansatzes liegt darin, dass der Roboterarm selbst als Objekt erfasst wird und permanent aus dem Bild herauskalkuliert (maskiert) werden muss, was den Programmieraufwand wiederum erhöht. Der wesentliche Vorteil liegt in der einfachen grundlegenden Implementation. Als kritischer Nachteil erweist sich jedoch die hohe Fehlerwahrscheinlichkeit, da transiente Umgebungsstörungen fälschlicherweise als Pseudoobjekte erkannt werden. Diese hohe Rate an Falsch-Positiv-Detektionen führt zu häufigen unplanmäßigen Anlagenstillständen, was die wirtschaftliche Gesamtanlageneffizienz (Overall Equipment Effectiveness, OEE) massiv senkt.

Eine selektivere Alternative stellt die ereignisbasierte Handerkennung dar. Hierbei detektiert ein spezielles, auf die Erkennung menschlicher Hände trainiertes KI-Modell die Extremitäten des Werkers. Sobald eine Hand oder Teile davon im Sichtfeld erfasst werden, wechselt das System in den definierten Sicherheitszustand. Diese Methode ist aufwendiger zu programmieren und benötigt eine signifikant höhere Rechenleistung (Schnittstellen für GPU/NPU). Sie arbeitet jedoch im industriellen Umfeld wesentlich zuverlässiger und minimiert die Erkennung von Pseudoobjekten. Dennoch besteht das Restrisiko einer fehlerhaften oder verzögerten Klassifikation (Falsch-Negativ-Rate). Zudem muss die Auslegung gemäß DIN EN ISO 13855 zwingend von der maximalen menschlichen Handgeschwindigkeit von 1600 mm/s bis 2000 mm/s ausgehen. KI-Modelle müssen daher eine entsprechend hohe Bildwiederholrate (Frames per Second, fps)

aufweisen, um diese Dynamik latenzfrei abzugreifen. Ein ungelöstes Problem bleibt die mathematische "Black-Box-Problematik" von Deep-Learning-Algorithmen, da diese schwer deterministisch zu verifizieren sind. Für sicherheitskritische Systeme nach ISO 13849-1 ist der lückenlose Nachweis einer fehlerfreien Entscheidung eine der größten aktuellen Forschungsherausforderungen. Zudem zeigt sich eine strikte Abhängigkeit von den Trainingsdatensätzen; weisen diese keine hinreichende Varianz bezüglich Hautfarben, Arbeitshandschuhen oder Handhaltungen auf, droht ein potenzieller Ausfall der Schutzfunktion.

Auf der sensorischen Ebene stehen sich physische und visuelle Systeme gegenüber. Die physische Sensorik realisiert die Raum- und Kontaktherausforderung über klassische Komponenten wie Laserschranken um den Sicherheitsbereich, kapazitive Sensorfelder (z.B. als sensitive Roboterhaut) oder induktive Felder, die auf die Annäherung von Körperteilen reagieren. Physische Sensoren sind kostengünstig und zeichnen sich durch eine einfache Problembearbeitung aus. Zudem eignen sie sich hervorragend als diversitäre, komplementäre Redundanz in Kombination mit optischen Systemen, um die strengen Anforderungen an die Ein-Fehler-Sicherheit nach DIN EN ISO 13849-1 [9] zu erfüllen.

Als Nachteil erweist sich die anspruchsvolle Kalibrierung, da die Sensorik sensibel auf elektromagnetische Umweltstörungen (EMV) reagiert und oft spezialisierte Auswerteelektronik benötigt. Da physische Sensoren zudem keine Tiefen- oder Richtungsinformationen liefern, können sie weder die Annäherungsgeschwindigkeit noch die genaue Position im Raum bestimmen. Ein zusätzlicher Faktor ist der mechanische Verschleiß sowie der hohe Montage- und Verkabelungsaufwand im rauen Industriealltag. Die visuelle Sensorik nutzt Kamerasysteme (2D-Systeme mit Kantenextraktion oder 3D-Time-of-Flight-Sensoren mit Tiefendaten) zur volumetrischen Raumüberwachung. Visuelle Systeme bieten den Vorteil einer zuverlässigen Klassifizierung spezieller Geometrien und erlauben Mehrfachanwendungen, indem dieselbe Hardware sowohl für Sicherheitsaufgaben als auch für bildverarbeitungsgestützte Greifprozesse (Pick-and-Place) genutzt werden kann. Eine laufende Neukalibrierung im Betrieb ist meist nicht notwendig.

Demgegenüber stehen hohe Anschaffungskosten und eine aufwendigere algorithmische Programmierung. Zudem reagieren optische Systeme empfindlich auf wechselnde Beleuchtungsverhältnisse, Schattenwurf oder Verschmutzungen der Linse. Normativ ist hierbei die Zertifizierung nach IEC 61496 zu beachten: Optische

Kamerasysteme für Sicherheitsanwendungen müssen zwingend als berührungslos wirkende Schutzeinrichtungen (ESPE) nach IEC 61496-1 zertifiziert sein. Standardmäßige Industriekameras ohne inhärente Hardware-Redundanz und gegenseitige Prozessorüberwachung sind für sicherheitsgerichtete Abschaltungen unzulässig.

Als finale Aktuatorik stehen der abrupte Nothalt und das adaptive Abbremsen zur Auswahl. Ein Nothalt führt bei einer Schutzfeldverletzung zum sofortigen Stillsetzen des Roboterarms, geregelt über eine kontrollierte oder unkontrollierte Energietrennung (Kategorie 0/1 nach DIN EN 60204-1). Im Sicherheitskontext von Robotersystemen wird diese Reaktion gemäß ISO 10218-1 präzise als sicherheitsgerichteter überwachter Halt (Safety-rated monitored stop) definiert. Sie bietet die höchste Sicherheitsstufe zur Risikominimierung. Für kooperative Arbeitsräume ist diese Reaktion jedoch ineffizient, da jede Annäherung des Werkers zu einem vollständigen Produktionstopp führt. Die abrupten Momentenmaxima verursachen zudem einen massiven mechanischen Verschleiß der Roboter Gelenke und Getriebe. Darüber hinaus reduziert die erhebliche Wiederanlaufzeit des Gesamtsystems (Bremsen lösen, Synchronisation der Servoregler) die Produktivität der Anlage signifikant. Eine wesentlich ergonomischere Lösung für die MRK stellt das Abbremsen des Roboterarms im Sinne des Speed and Separation Monitoring (SSM) dar. Der Manipulator reduziert seine Bahngeschwindigkeit in Abhängigkeit vom Abstand zum Menschen stufenweise oder kontinuierlich über vordefinierte Geschwindigkeitszonen. Diese Funktion der sicheren Geschwindigkeitsreduzierung (Safely-Limited Speed, SLS) sichert fließende Prozesse, minimiert den mechanischen Verschleiß und muss über sichere Antriebsregler (z.B. gemäß IEC 61800-5-2) validiert werden, um die funktionale Sicherheit zu garantieren.

Der Ansatz erfordert jedoch einen hohen softwareseitigen Aufwand für die Bildverarbeitung und verlangt bei Fehlfunktionen in der Regelung eine konsequente zweikanalige Überwachung. Zu beachten bleibt das regelungstechnische Restrisiko durch Trägheit: Aufgrund der Eigenmasse des Manipulators besitzt dieser auch während des Verzögerungsvorgangs eine hohe kinetische Energie. Das Sicherheitskonzept muss mathematisch garantieren, dass die Drosselung schnell genug erfolgt, bevor der Mensch die nächste kritische Distanzschwelle erreicht. Da der Roboter auch bei geringen Geschwindigkeiten hohe statische Kräfte aufbauen kann, muss das System so ausgelegt sein, dass ein Quetschen von Körperteilen zwischen dem Roboter und starren Umgebungsstrukturen physikalisch oder algorithmisch vollständig ausgeschlossen ist.

5.1.2 Gegenüberstellung spezialisiertes KI-Modell und einfacher Objekterkennungsalgorithmus

5.2 Befehlspipeline

5.2.1 Vergleich von theoretischer und realer Performance

6 Fazit & Ausblick

Literaturverzeichnis

- [1] Fraunhofer-Institut für Fabrikbetrieb und -automatisierung IFF, “Projektions- und kamerabasierte sicherheitstechnologien,” <https://www.iff.fraunhofer.de/de/geschaeftsbereiche/robotersysteme/projektions-und-kamerabasierte-technologien.html>, 2026, abgerufen am: 11. Juni 2026.
- [2] M. Bdiwi, M. Pfeifer, and A. Sterzing, “A new strategy for ensuring human safety during various levels of interaction with industrial robots,” *CIRP Annals*, vol. 66, no. 1, pp. 453–456, 2017.
- [3] F. Graziola, “Entwicklung eines prototypen einer passiv-infrarot schutzeinrichtung zur erkennung der hand-arm region bei sägearbeiten an einer kreissäge,” Diplomarbeit, Fachhochschule Bonn-Rhein-Sieg, Fachbereich Elektrotechnik, Maschinenbau und Technikjournalismus, Sankt Augustin, Aug. 2003.
- [4] W. Schulz, “Anwendung und vergleich von algorithmen zur sicherheitsgerichteten auswertung von 2d laserscanner-entfernungsdaten,” Sankt Augustin, 2005.
- [5] N. Mandischer, C. Weidemann, M. Hüsing, and B. Corves, “Berührungslose sicherheit für serielle manipulatoren durch optische eintrittsdetektion mit einem bewegten 3d-kamerasensor,” in *Tagungsband des Robotik-Kongresses (oder entsprechende Konferenz/Fachtagung)*. Aachen, Deutschland: Institut für Getriebetechnik, Maschinendynamik und Robotik (IGMR), RWTH Aachen University, 2022, pp. 1–8.
- [6] S. K. Singh, D. S. Chauhan, M. Vatsa, and R. Singh, “A robust skin color based face detection algorithm,” *Tamkang Journal of Science and Engineering*, vol. 6, no. 4, pp. 227–234, 2003.
- [7] Europäisches Parlament und Rat der Europäischen Union, “Richtlinie 2006/42/eg vom 17. mai 2006 über maschinen und zur Änderung der richtlinie 95/16/eg (neufassung),” Amtsblatt der Europäischen Union, L 157/24, 2006, gültig bis zum 19. Januar 2027.
- [8] —, “Verordnung (eu) 2023/1230 vom 10. mai 2023 über maschinen sowie zur aufhebung der richtlinie 2006/42/eg des europäischen parlaments und des rates und der richtlinie 73/361/ewg des

rates,” Amtsblatt der Europäischen Union, L 165/1, 2023, verbindlich ab dem 20. Januar 2027.

- [9] DIN Deutsches Institut für Normung e. V., “Sicherheit von maschinen – sicherheitsbezogene teile von steuerungen – teil 1: Allgemeine gestaltungsleitsätze (iso 13849-1:2023); deutsche fassung en iso 13849-1:2023,” Beuth Verlag GmbH, Berlin, Norm DIN EN ISO 13849-1, Dec. 2023.
- [10] —, “Industrieroboter – sicherheitsanforderungen – teil 1: Roboter (iso 10218-1:2011); deutsche fassung en iso 10218-1:2011,” DIN Media GmbH, Berlin, Norm DIN EN ISO 10218-1, Jan. 2012.
- [11] —, “Industrieroboter – sicherheitsanforderungen – teil 2: Robotersysteme und integration (iso 10218-2:2011); deutsche fassung en iso 10218-2:2011,” DIN Media GmbH, Berlin, Norm DIN EN ISO 10218-2, Jun. 2012.
- [12] —, “Roboter und robotikgeräte – kollaborierende roboter (iso/ts 15066:2016),” DIN Media GmbH, Berlin, Vornorm DIN ISO/TS 15066*DIN SPEC 5306, Apr. 2017.
- [13] DIN Deutsches Institut für Normung e. V. und VDE Verband der Elektrotechnik Elektronik Informationstechnik e. V., “Sicherheit von maschinen – berührungslos wirkende schutzeinrichtungen – teil 1: Allgemeine anforderungen und prüfungen (iec 61496-1:2020); deutsche fassung en iec 61496-1:2020,” VDE Verlag GmbH / DIN Media GmbH, Berlin, Norm DIN EN IEC 61496-1 (VDE 0113-201), Jun. 2021.
- [14] DIN Deutsches Institut für Normung e. V., “Sicherheit von maschinen – anordnung von schutzeinrichtungen im hinblick auf annäherung des menschlichen körpers (iso 13855:2024); deutsche fassung en iso 13855:2024,” DIN Media GmbH, Berlin, Norm DIN EN ISO 13855, Oct. 2025.
- [15] Microsoft Developers. Task parallel library tpl. [Online; Zugriffen am 10. Juni 2026]. [Online]. Available: <https://learn.microsoft.com/de-de/dotnet/standard/parallel-programming/task-parallel-library-tpl>
- [16] S. Cleary, *Concurrency in CSharp Cookbook*, 2nd ed. O’Reilly Media, Inc., 2019.
- [17] Microsoft Developers. asynchrone programmiermuster. [Online; Zugriffen am 10. Juni 2026]. [Online].

Available: <https://learn.microsoft.com/de-de/dotnet/standard/asynchronous-programming-patterns/>

- [18] M. Electric. Mitsubishi movemaster ex rv-m1 manual. [Online; Zugegriffen am 10. Juni 2026]. [Online]. Available: https://www.roboex.com/Mitsubishi%20Movemaster%20Robot%20Manual/RV-M1/RV-M1_InstructionManual.pdf
- [19] R. Meta, “Event-driven programming for responsive robotic systems,” <https://roboticsmeta.com/event-driven-programming-for-responsive-robotic-systems/>, 2024, accessed: 2026-06-19.

Abkürzungsverzeichnis

CLR	Common Language Runtime
CMOS	Complementary Metal-Oxide-Semiconducto
Cobot	Collaborative Robot
CTS	Clear to Send
DFT	Diskrete Fouriertransformation
DH	Denavit Hartenberg
DIN	Deutsches Institut für Normung
DLL	Dynamic Link Library
DSR	Data Set Ready
DTR	Data Terminal Ready
DU	Drive Unit
EN	Europäische Norm
ER	Error Read
FG	Frame Ground
FOV	Field of View
fps	frames per second (Bilder pro Sekunde)
IPK	Institut für Produktionsanlagen und Konstruktionstechnik
ISO	International Organization for Standardization
JSON	JavaScript Object Notation
KI	Künstliche Intelligenz
LED	Leuchtdiode
LIDAR	Light Detection and Ranging
MC	Move Continuous
ML	Maschinelles Lernen
MO	Move
ms	Millisekunde
Mutex	Mutual Exclusions
MVC	Model View Controller
NC	Not Connected
P/Invoke	Platform Invocation Services
PCL	Point Cloud Library
PL	Performance Level
RANSAC	Random Sample Consensus Algorithmus
RXD	Receive Data
ROR	Radius Outlier Removal
RS-232	Recommended Standard 232
SCARA	Selective Compliance Assembly Robot Arm
SOR	Statistical Outlier Removal

SP	Speed
SSM	Speed and Separation Monitoring
TCP	Tool Center Point
TCP/IP	Transmission Control Protocol/Internet Protocol
ToF	Time of Flight
TXD	Transmit Data
WH	Where
2D	Zweidimensional
3D	Dreidimensional

Abbildungsverzeichnis

2.1	Testbild Passiv-Infrarot Schutzeinrichtung an einer Kreissäge aus [3]	5
2.2	Problematik von blockierenden Abläufen	9
3.1	Sequenzielle Abfolge an Befehlen (Eigene Darstellung) . .	11
3.2	Messexperiment zur Evaluation der Sortierperformance .	13
3.3	Befehle innerhalb einer Objektsortierungssequenz	14
4.1	Gegenüberstellung von blockierenden synchronen Abläufen und dem ressourceneffizienten, asynchronen Taskbasierten Muster (TAP) bei E/A-Operationen.	20
5.1	Projektions- und kamerabasierte Technologie zur Arbeitsraumüberwachung	23

Erklärungen zur Arbeit und Hilfsmitteln

Verwendete Hilfsmittel

Für die Erstellung dieser Studienarbeit wurden folgende Tools eingesetzt:

- **Hardware:** SICK Visionary-T Mini-CX, Mitsubishi Movemaster EX-RV M1, uEye Industriekamera
- **Software:** OpenCV Bibliothek, Visual Studio (C++ und C#), RoboETH, drawIO (Erstellung von Grafiken), texStudio, uEye-Cockpit und IDS Kameramanager, GitHub (Kollaboratives Arbeiten und Versionsverwaltung der LaTeX-Vorlage).

KI-Tools und relevanter Einsatzbereich

Dies erfolgt im Einklang mit der entsprechenden Verordnung der DHBW (Weblink).

Tool	Art der Nutzung	Abschnitt
Google Gemini AI Student Pro License	Formatierung von LaTeX-Code, Erstellung von Blankovorlagen für Tabellen und Listen	gesamte Arbeit
Google Gemini AI Student Pro License: Deep Search-Feature	Tiefgreifende Analyse des Datenblatts des Movemaster EX RV-M1: Finden von Abschlusszeichen im RS232-String, Pin-Belegung für Hardwaregestütztes Acknowledgement über RS232, Physikalische Signalanalyse für die Implementierung einer Flow-Control; Ergebnis der "Deep Search" kann jederzeit als PDF-Dokument bereitgestellt werden	4.3.2
DeepL AI Übersetzer	Formulierung des englischsprachigen Abstracts	Abstract

Aufteilung der Kapitel

Die Erstellung der vorliegenden Studienarbeit erfolgte in gemeinsamer Kooperation. In der folgenden Tabelle ist dokumentiert, welcher Autor die primäre Urheberschaft für die jeweiligen Kapitel bzw. Abschnitte trägt:

Kapitel / Abschnitt	Kapitelnummer	Autor
Einleitung	1	Gemeinsame Bearbeitung
Robotik	??	Luis Magnus Thiel
Industrielle Bildverarbeitung	??	Adrian Sommer
Grundlagen der Informatik	??	Luis Magnus Thiel
Hardware	??	Luis Magnus Thiel
Software	??	Adrian Sommer
Messreihe	??	Luis Magnus Thiel
Optimierung der Softwarestruktur	??	Luis Magnus Thiel
Optimierung der Bildgewinnung	??	Luis Magnus Thiel
Optimierung und Verbesserungsmöglichkeiten an der Software	??	Adrian Sommer
Normative Anforderungen	??	Gemeinsame Bearbeitung
Umsetzbare Sicherheitsanforderungen	??	Gemeinsame Bearbeitung
Ansatz Geschwindigkeits- und Abstandsüberwachung	??	Adrian Sommer
Fazit und Ausblick	??	Gemeinsame Bearbeitung

Hiermit bestätigen wir, Luis Magnus Thiel und Adrian Sommer,
dass die oben gemachten Angaben der Wahrheit entsprechen.

Horb, den 19. Juni 2026



Ort, Datum

Unterschrift (Luis Magnus Thiel)

Horb, den 19. Juni 2026



Ort, Datum

Unterschrift (Adrian Sommer)

Anhang

Inhalt des digitalen Anhangs

- Anleitung zur Programmaufsetzung (Luis Thiel)
- gesamter LaTeX-Ordner
- MATLAB-Livescript für die Berechnung der Objekt-Verfahrzeiten
- gesamte Visual-Studio-Projektmappe

Sequenz	Beschreibung	Zeit [s]
1	Startposition -> Objekt 1 + GC	4,793 + 0,5
1	Objekt 1 -> Objekt 1- <i>Z_{offset}</i>	0,682
1	Objekt 1- <i>Z_{offset}</i> -> Magazin 1- <i>Z_{offset}</i>	1,555
1	Magazin 1- <i>Z_{offset}</i> -> Magazin 1 + GO	0,697 + 1
1	Magazin 1 -> Magazin 1- <i>Z_{offset}</i>	0,697
2	Magazin 1- <i>Z_{offset}</i> -> Objekt 2 + GC	2,195 + 0,5
2	Objekt 2 -> Objekt 2- <i>Z_{offset}</i>	0,676
2	Objekt 2- <i>Z_{offset}</i> -> Magazin 2- <i>Z_{offset}</i>	1,586
2	Magazin 2- <i>Z_{offset}</i> -> Magazin 2 + GO	0,638 + 1
2	Magazin 2 -> Magazin 2- <i>Z_{offset}</i>	0,638
2	Magazin 2- <i>Z_{offset}</i> -> Objekt 3 + GC	1,160 + 0,5
3	Objekt 3 -> Objekt 3- <i>Z_{offset}</i>	0,677
3	Objekt 3- <i>Z_{offset}</i> -> Magazin 3- <i>Z_{offset}</i>	0,886
3	Magazin 3- <i>Z_{offset}</i> -> Magazin 3 + GO	0,647 + 1
3	Magazin 3 -> Magazin 3- <i>Z_{offset}</i>	0,647
4	Magazin 3- <i>Z_{offset}</i> -> Objekt 4 + GC	0,915 + 0,5
4	Objekt 4 -> Objekt 4- <i>Z_{offset}</i>	0,634
4	Objekt 4- <i>Z_{offset}</i> -> Magazin 4- <i>Z_{offset}</i>	0,918
4	Magazin 4- <i>Z_{offset}</i> -> Magazin 4 + GO	0,649 + 1
4	Magazin 4 -> Magazin 4- <i>Z_{offset}</i>	0,649
-	Magazin 4- <i>Z_{offset}</i> -> Startposition	4,467
		31,88

Tabelle 6.1: Theoretische Ausführungszeiten der Befehlspipeline